DAY 06 · DEBUGGING

unittest, pdb & Debugging

unittest, pdb, and a repeatable method for maximizing passing tests.

2–3 hours / day06_debugging_unittest / Debug method

INSIDE THIS BOOKLET

unittest pdb Triage Workflow

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day06_debugging_unittest/` · the course runner `python run.py 6`

In this session

Why a method beats talent 3

1 unittest in depth 3

2 Print debugging, done well 7

3 pdb: pausing the program to look around 8

4 The exam workflow (practise this exactly) 11

5 Scratch scripts and differential testing 12

6 What a failure tells you 13

7 The toolbox: Python you'll meet in today's exercises 13

Exercises 16

End-of-day checklist 17

> **◎ Today's goal**
>
> Have a repeatable process for "here's broken code and a test suite, maximise passing tests in 2h". Today is about *method*, not syntax.

Why a method beats talent

Picture the exam. You open a 200-line file, run the tests, and 23 of 40 fail. Where do you start? Without a plan, most people do something like this: they read the first failing test, jump into the code, change something that looks suspicious, run the tests again, see a different number, change something else... After an hour they're not sure which of their changes helped, one of them broke a test that used to pass, and they've spent forty minutes on one function. With a plan, you work like a detective. You find out which pieces of code the failures have in common. You pick the simplest failing case, work out by hand what the answer *should* be, and watch the program step by step until the moment its state stops matching yours. That moment points at the bug. You fix one line, re-run *everything*, and check the score went up. Then you repeat. Today gives you the three tools that method needs, and then the method itself:

1. **unittest**, in more depth than Day 1, so you can read any test file and run exactly the tests you want.
2. **print debugging and pdb**, the two ways of seeing what's happening inside a running program.
3. **scratch scripts**, for trying things out quickly.

1 unittest in depth

You don't write tests in the exam, but you must *read* them quickly. Each test is the most precise statement there is of what the code must do: an exact input and an exact expected output. Here's a test file using all the features you're likely to meet. We'll go through them one by one afterwards.

```
import unittest

class TestThing(unittest.TestCase):
    maxDiff = None              # show FULL diffs of long lists/strings (default truncates!)

    def setUp(self):             # runs before EACH test method
        self.t = Tokenizer()

    def tearDown(self): ...      # after each test

    def test_something(self):    # only methods starting with test_ are run
        self.assertEqual(a, b)
        self.assertNotEqual(a, b)
        self.assertTrue(x); self.assertFalse(x)
        self.assertIs(a, b); self.assertIsNone(x); self.assertIsNotNone(x)
        self.assertIn(x, container); self.assertNotIn(x, container)
        self.assertIsInstance(obj, cls)
        self.assertAlmostEqual(0.1 + 0.2, 0.3)          # 7 decimal places
        self.assertCountEqual([1, 2, 2], [2, 1, 2])      # same elements, any order
        self.assertLess(a, b); self.assertGreaterEqual(a, b)
        with self.assertRaises(ValueError):
            f()
        with self.assertRaises(KeyError) as ctx:
            g()
```

```
        self.assertEqual(ctx.exception.args[0], "x")
        with self.assertRaisesRegex(ValueError, "line 3"):
            h()

    def test_many(self):
        for inp, exp in [(1, 2), (2, 4)]:
            with self.subTest(inp=inp):      # failures are reported per-case, loop continues
                self.assertEqual(double(inp), exp)
```

1.1 `setUp` and `tearDown`: a fresh start for every test

If a class has a method called `setUp`, unittest runs it **before each test method**. It's usually used to build the object being tested:

```
class TestCache(unittest.TestCase):
    def setUp(self):
        self.cache = LRUCache(capacity=2)

    def test_put_then_get(self):
        self.cache.put("a", 1)
        self.assertEqual(self.cache.get("a"), 1)

    def test_empty(self):
        self.assertEqual(len(self.cache), 0)
```

Because `setUp` runs again before `test_empty`, that test gets a brand-new, empty cache. Whatever `test_put_then_get` did doesn't matter. That's on purpose: each test should stand on its own. Practical tip: when a failing test uses `self.something`, **look at `setUp`** to see how that object was created. The capacity, the vocabulary size and the initial data set up there are part of the test's input. `tearDown` is the opposite: it runs *after* each test, for cleanup. You'll rarely see it in exam tests.

1.2 The assertions, one by one

Each assertion checks one condition. If the condition is false, the test fails and prints a message showing the values involved. Here they are with what they check and a typical use:

Assertion	Passes when	Typical use
`assertEqual(a, b)`	`a == b`	the workhorse: "the function returns this"
`assertNotEqual(a, b)`	`a != b`	"these two encodings must differ"
`assertTrue(x)` / `assertFalse(x)`	`x` is truthy / falsy	`assertTrue(is_valid(s))`
`assertIs(a, b)`	`a` and `b` are the *same object*	"returns the same list, not a copy"
`assertIsNone(x)`	`x is None`	"`get` of a missing key gives `None`"
`assertIn(x, c)`	`x in c`	"the key is present"
`assertIsInstance(obj, cls)`	`obj` is of type `cls`	"decode returns a `str`, not `bytes`"
`assertAlmostEqual(a, b)`	`round(a - b, 7) == 0`	comparing floats (Day 1)
`assertCountEqual(a, b)`	same elements, same counts, *any order*	when the order isn't specified
`assertLess(a, b)` etc.	`a < b` etc.	"the compressed version is shorter"

Two of them deserve a closer look. **`assertAlmostEqual`** exists because floats are approximate (Day 1, section 2). It rounds the *difference* to 7 decimal places and checks that it's zero:

```
self.assertAlmostEqual(average([0.1, 0.2]), 0.15)              # passes, despite
    0.15000000000000002
self.assertAlmostEqual(price_with_tax(10), 12.1, places=2)    # only check 2 decimal places
```

`assertCountEqual` has a misleading name. It doesn't compare counts of something. It checks that both sides contain the same elements, the same number of times, in any order. When you see it in a test, the spec doesn't fix the order of the result.

```
self.assertCountEqual(group_members(), ["ana", "ben"])   # ["ben", "ana"] also passes
```

1.3 Testing that errors are raised

On Day 1 you saw `with self.assertRaises(ValueError):`. It passes only if the code inside the `with` block raises a `ValueError` (or a subclass of it, Day 4). There are two more forms. With `as ctx`, you can examine the exception after it's been raised:

```
with self.assertRaises(KeyError) as ctx:
    store.get_strict("missing")
self.assertEqual(ctx.exception.args[0], "missing")      # the error carried the key
```

`ctx.exception` is the exception object, and `args[0]` is the first value it was created with, usually the message. `assertRaisesRegex` also checks the error *message*, matching it against a pattern. When a spec says "the error message must mention the line number", this is how it's tested:

```
with self.assertRaisesRegex(ValueError, "line 3"):
    parse("ok\nok\nbroken")
```

So if the code raises the right exception but with a different message, for example `"line 2"` because of a 0-based/1-based mix-up, this test fails.

1.4 `subTest`: many cases in one test

Sometimes a test checks the same function on a whole list of inputs:

```
def test_many(self):
    for inp, exp in [(1, 2), (2, 4), (3, 6)]:
        with self.subTest(inp=inp):
            self.assertEqual(double(inp), exp)
```

Without `subTest`, the first failing case would stop the test, and you'd never learn about the others. With it, every case runs, and each failure is reported separately with its parameters:

```
FAIL: test_many (test_x.TestDouble.test_many) (inp=3)
AssertionError: 7 != 6
```

This is valuable information. If cases 1 and 2 pass and only 3 fails, the bug is something specific to that input. The *pattern* of which inputs fail is often the biggest clue you'll get.

1.5 `maxDiff = None`

When two long lists or strings differ, unittest normally cuts the difference report short, with a message like "Diff is 1520 characters long. Set self.maxDiff to None to see it." Adding `maxDiff = None` to the test class shows everything. You can add it temporarily to an exam test class while investigating. It changes only how much is printed, not what's checked.

1.6 Running exactly the tests you want

From the folder that contains the test files:

```
python -m unittest                                  # discover test*.py in cwd
python -m unittest -v                               # one line per test
python -m unittest -f                               # stop at first failure
python -m unittest test_bpe                         # one file (module name, no .py)
python -m unittest test_bpe.TestEncode              # one class
python -m unittest test_bpe.TestEncode.test_empty   # one test
python -m unittest -k encode                        # tests whose name contains "encode"
python -m unittest -v 2>&1 | grep -E "FAIL|ERROR"   # list of failing tests only
python -m unittest 2>&1 | tail -3                   # just the summary
```

When you'd use each:

- The plain run gives you your **score**. Do it at the start, and after every fix.
- `-v` gives you the **map**: every test by name, marked `ok`, `FAIL` or `ERROR`. Do it once at the start, to see which areas are broken.
- The dotted names (`file.Class.test`) let you **focus** on one test while you work on it. The output stays short, and the run is quick.
- `-k encode` runs every test with "encode" in its name, which is handy for working on one feature.
- The `grep` line prints only the names of failing tests, a compact to-do list.
- The `tail -3` line shows just the last three lines, i.e. the summary.

About the `2>&1 | ...` part: a program can print to two output channels, "standard output" and "standard error". unittest writes its report to standard error. The `|` symbol sends output into another command (like `grep` or `tail`), but only standard output. `2>&1` merges standard error into standard output so the pipe gets everything. You don't need to understand the details. Just copy the line as it is.

1.7 Reading the summary

At the end of each run, unittest prints a summary like:

```
Ran 40 tests in 0.031s

FAILED (failures=3, errors=5)
```

Reading the summary: `FAILED (failures=3, errors=5)` — **errors** are exceptions raised in your code (crashes: `TypeError`, `KeyError`, `IndexError`...), **failures** are wrong answers. Errors are often the cheapest to fix (the traceback points at the line).

1.8 Reading a traceback

When a test has an ERROR, unittest prints a **traceback**: the chain of function calls that led to the crash, oldest first. Here's an example:

```
ERROR: test_encode_word (test_bpe.TestEncode.test_encode_word)
Traceback (most recent call last):
  File "test_bpe.py", line 88, in test_encode_word
    ids = self.tok.encode("hello")
  File "bpe.py", line 120, in encode
    return self.encode_ordinary(text)
  File "bpe.py", line 101, in encode_ordinary
    ids = merge(ids, pair, self.merges[pair])
KeyError: (104, 101)
```

Read it as a story: the test called `encode`, which called `encode_ordinary`, which crashed on line 101 with a `KeyError` because the pair `(104, 101)` wasn't in `self.merges`.

> **Read the last frame in your file**
>
> **The last line of the traceback inside YOUR file is where to look**, not the test line.

Here that's `bpe.py`, line 101. That's where the problem *showed up*. It isn't always where the bug *is*: perhaps an earlier line chose a pair that shouldn't have been chosen. But it's the right place to start reading, and you work backwards from there.

2 Print debugging, done well

2.1 The idea

The simplest way to see inside a running program is to make it print its variables. It works everywhere, needs no tools, and is enough for a large share of bugs. Done *well*, it means printing the right values, clearly labelled, in a form that shows exactly what they are.

```
print(f"{i=} {cp=:#x} {data[i:i+4]!r}")      # self-documenting f-strings
print(f"{cp:021b}")                           # binary view for bit-twiddling code
print("DEBUG", repr(s))                       # repr shows '\n', spaces, quotes, b'' prefixes
import sys; print(..., file=sys.stderr)       # stderr won't get mixed with expected stdout
```

What each line gives you:

- `f"{i=}"` prints `i=3`, so you never have to wonder which number is which. You can combine `=` with a format, as in `{cp=:#x}`, which prints `cp=0xe9`.
- `f"{cp:021b}"` shows a number as 21 binary digits. For bit-manipulation code (Day 5), wrong bits are visible at a glance.
- `repr(s)` shows values *precisely*: strings in quotes, invisible characters like `\n` written out, and bytes with their `b` prefix. Many "these look identical but the test says they differ" mysteries are solved by `repr`. One side turns out to be `'a '` (with a trailing space), or `'1'` (a string) instead of `1` (a number), or `b'x'` instead of `'x'`.
- `file=sys.stderr` sends the output to the error channel, which matters only if the code being tested prints things itself and a test checks that output.

2.2 Where to put the prints

- unittest **shows** prints from tests (in the output stream), so prints work fine; just remove/disable them afterwards because they clutter output. A `DEBUG = True` flag + `if DEBUG: print(...)` is handy.
- Put the print **just before** the line that crashes or the return that gives a wrong value.

Choosing *where* to print is the real skill. Print at the inputs of the function you suspect, and inside its main loop, showing the state on each pass. Then compare each printed line with what you worked out by hand. The first line that disagrees tells you that the bug is in the code that ran just before it.

3 pdb: pausing the program to look around

3.1 What a debugger is

Printing has a limitation: you have to decide in advance what to print. If what you printed doesn't explain the bug, you add more prints and run again. A **debugger** works differently. It *pauses* the program at a line of your choice, and while it's paused you can look at any variable, evaluate any expression, and then run the program forward one line at a time, watching what changes. Python's built-in debugger is called **pdb**, and the exam instructions point to it explicitly.

3.2 Setting a breakpoint

You pause the program by adding this line where you want to stop:

```
breakpoint()            # Python 3.7+, same as: import pdb; pdb.set_trace()
```

When execution reaches this line, the program stops and shows the line it's about to run, followed by a `(Pdb)` prompt. All the local variables at that point are available to you. Inside a loop, an unconditional breakpoint would stop on *every* pass, which is tedious if the loop runs a thousand times. Put it inside an `if` so it only triggers in the case you care about: Conditional (only for the interesting case — essential inside loops):

```
if i == 7 or cp > 0x10FFFF:
    breakpoint()
```

Run a test file and the debugger stops at `breakpoint()`:

```
python -m unittest test_ex02_intervals.TestMerge.test_touching   # run ONLY the test you
    care about
```

Run just that one test. Otherwise every other test that passes through the same line stops there too.

3.3 A complete example session

Here's a small buggy function, followed by a real pdb session investigating it. The spec says `count_vowels("banana")` should return 3.

```python
def count_vowels(word):
    count = 0
    for ch in word:
        breakpoint()
        if ch in "aeiou":
            count =+ 1
    return count
```

```
> example.py(5)count_vowels()
-> if ch in "aeiou":
(Pdb) p ch, count
('b', 0)
(Pdb) c
> example.py(5)count_vowels()
-> if ch in "aeiou":
(Pdb) p ch, count
('a', 0)
(Pdb) n
> example.py(6)count_vowels()
-> count =+ 1
(Pdb) n
> example.py(3)count_vowels()
-> for ch in word:
(Pdb) p count
1
(Pdb) c
> example.py(5)count_vowels()
-> if ch in "aeiou":
(Pdb) c
> example.py(5)count_vowels()
-> if ch in "aeiou":
(Pdb) p ch, count
('a', 1)
(Pdb) n
(Pdb) n
(Pdb) p count
1
```

What we did:

1. The program stopped at the `breakpoint()`. The line starting with `->` is the *next* line to run.
2. `p ch, count` printed the variables: first letter `'b'`, count 0. Correct so far.
3. `c` (continue) ran on until the breakpoint was hit again, now for `'a'`.
4. `n` (next) ran one line at a time. After the `count =+ 1` line, `count` was 1. That's fine for the first vowel.
5. On the *second* `'a'`, count was still 1 after the same line. That's the divergence: it should have become 2.

Looking closely at the line: `count =+ 1` is `count = +1`, which sets count to plus one every time. The intended line was `count += 1`. (Remember to delete the `breakpoint()` afterwards.)

3.4 The commands

Commands at the `(Pdb)` prompt:

Cmd	Meaning
`l` / `ll`	list code around current line / the whole current function
`n`	next line (step **over** calls)
`s`	step **into** a call
`r`	continue until the current function returns
`c`	continue until the next breakpoint
`unt N`	continue until line N (great to skip a loop)
`p expr` / `pp expr`	print / pretty-print an expression
any expression	e.g. `len(ids)`, `ids[i:i+3]`, `sorted(stats.items())[:5]`
`w`	where: the stack trace
`u` / `d`	move up / down the stack (inspect the caller's variables, e.g. the test's inputs)
`b 42` / `b file.py:42` / `b 42, i == 3`	set (conditional) breakpoint
`cl`	clear breakpoints
`display expr`	auto-print expr every time execution stops
`interact`	full Python REPL with the current variables (Ctrl-D to return)
`q`	quit
Enter	repeat last command

Most sessions use only a handful of these, in roughly this order:

1. **Get your bearings** with `ll`, which shows the whole current function with an arrow at the current line.
2. **Look** at variables with `p`, or just type any expression, like `len(ids)` or `ids[i:i+3]`, and pdb evaluates it right there.
3. **Move forward.** `n` runs the current line and stops at the next one. If the current line calls a function you suspect, `s` steps *into* that function instead of running it in one go. `r` runs until the current function returns, and `c` runs until the next breakpoint.
4. **Look at the bigger picture.** `w` shows the chain of calls that led here, like a traceback. `u` moves up one level, to the function that called this one, so you can see *its* variables, for example the exact input the test passed in. `d` moves back down.
5. `q` quits when you've found what you need.

Three practical tips:

- `display count` makes pdb print `count` automatically every time it stops, which is ideal for watching a value change as you press `n`.
- Pressing Enter on an empty line repeats the last command, so you can press `n` once and then just keep hitting Enter.
- If one of your variables is called `n`, `c`, `s`, `l` or `p`, typing its name runs the *command* instead. Use `p n` to print it.

3.5 Post-mortem: examining a crash after it happens

Post-mortem (inspect the state at the moment of a crash):

```
python -m pdb -c continue my_script.py      # runs; on an uncaught exception you land at the
    crash
```

```
import pdb, traceback
try:
    f()
except Exception:
    pdb.post_mortem()                        # inside a scratch script
```

"Post-mortem" means "after death". Instead of setting a breakpoint beforehand, you let the program crash, and pdb opens at the exact line of the crash, with every variable still holding the value it had at that moment. For an `IndexError`, typing `p i, len(data)` usually explains the problem instantly. The first form does this for a whole script. The second does it for one function call inside a script you wrote.

> **Post-mortem inside unittest**
>
> Note: unittest catches exceptions, so post-mortem doesn't trigger from `python -m unittest`. Write a tiny scratch script that calls the function with the failing test's inputs instead (see section 5).

`pdb++` (`pip install pdbpp`) is a drop-in upgrade (colours, `sticky` mode). Nice at home; don't depend on it.

4 The exam workflow (practise this exactly)

This is the routine to make automatic. First the steps, then why each one is there.

1. **Read (≈10 min).** Skim the whole spec once. Skim the implementation: what are the functions/classes, what calls what? Skim the test file names/classes. Don't fix anything yet.
2. **Run everything.** `python -m unittest 2>&1 | tail -3`. Note the score.
3. **Triage.** `python -m unittest -v 2>&1 | grep -E "ERROR|FAIL"`. Group failures by the function under test. Start at the **bottom of the call graph**: helpers that everything else uses (e.g. `get_stats`, `merge`, `parse_amount`). One fix there can turn 15 tests green.
4. **Loop, one bug at a time:**
   - pick ONE failing test, ideally the simplest (smallest input),
   - reproduce it in isolation (`python -m unittest file.Class.test`),
   - compute the expected result *by hand* from the spec for that small input,
   - find the first point where the program's state diverges from your hand computation (print / pdb),
   - fix the minimal thing, **re-run the whole suite** (did the score go up? did anything else break?).
5. **Don't get stuck.** If a test resists > 10–15 min, write it down and move on. Come back later — other fixes often make it clear.
6. **Spec vs. tests.** When they disagree, the tests win. Look at the test to learn the exact expected format (spaces, ordering, exception type, rounding).
7. **Last 10 min:** remove debug prints / breakpoints, run the full suite one final time.

Why read first? Ten minutes of reading gives you a map: which function does what, and which ones depend on which. Without it, you'll "fix" a symptom in a high-level function when the real bug is in a

helper, or change code that was actually correct. **Why note the starting score?** So you can tell whether each change helped or hurt. **What's a "call graph", and why start at the bottom?** A call graph is simply "which function calls which". Suppose `encode` calls `merge`, and `merge` is broken. Then every test of `encode` fails too, even though `encode` itself may be fine. If you fix `merge` first, many failures disappear at once, and you avoid wasting time investigating failures that are only side effects. So start with the functions that don't call anything else (the bottom of the graph), and work upward. **Why the simplest failing test?** Because you need to work out the correct answer by hand, and that's only practical for small inputs. Your hand computation is the yardstick. You run the program, compare its state against your own working at each step, and the bug lies between the last point where they agreed and the first point where they don't. **Why re-run everything after each fix?** Two reasons. Your fix might be wrong, or right but break something else. And several bugs can hide each other: fixing one sometimes doesn't raise the score because another bug still breaks the same tests. Re-running everything shows you what really changed. **Why move on when stuck?** Your score is the number of passing tests, not the effort spent. Fifteen minutes on one stubborn test could fix three easier ones elsewhere. And often a later fix makes the stubborn one obvious. **Why do the tests win over the spec?** Because the tests are what's graded. If the spec is vague about a format, the test that checks it shows you exactly what's wanted. **Why clean up at the end?** A leftover `breakpoint()` makes the grader's test run sit waiting for input forever. A leftover print can break tests that check printed output.

5 Scratch scripts and differential testing

5.1 Scratch scripts

A **scratch script** is a throwaway file where you call the function you're investigating directly, with whatever inputs you like, and print the results:

```
from ledger import parse_amount
for s in ["150.25", "150.5", "-3.10", "0.05"]:
    print(s, parse_amount(s))
```

Iterating on a 4-line scratch script is faster than re-running the suite. Save it as `scratch.py` next to the code and run it with `python scratch.py`. It lets you try many inputs in one go, including edge cases the tests may not have, and it's where post-mortem pdb works (which unittest prevents). It isn't graded, so it doesn't matter whether you delete it.

5.2 Differential testing: compare with a simple version

Differential testing

If you can write a slow-but-obviously-correct version of a helper (or use a built-in like `bytes.decode`, `sorted`, `str.split`) compare outputs on many inputs:

```
import random
for _ in range(1000):
    xs = [random.randint(0, 9) for _ in range(random.randint(0, 6))]
    assert my_func(xs) == obviously_correct(xs), xs
```

Minimal failing input

The first assertion that fires gives you a minimal failing input.

Here's the idea. Exam code is usually written to be *fast*, with clever index tricks and incremental updates. But for many functions, a slow version is easy to write: two nested loops, or a call to a built-in. For example, "the largest sum of `k` consecutive numbers" can be computed slowly as `max(sum(xs[i:i+k]) for i in range(len(xs) - k + 1))`. So you generate hundreds of small random inputs and compare the clever version with the simple one. The `, xs` at the end of the `assert` line makes Python print the input that failed. Because the random inputs are tiny (at most 6 numbers from 0 to 9), the failing input is small enough to trace by hand. Several tests in this course are written exactly this way. You'll see names like `test_random_against_brute_force`.

6 What a failure tells you

Each kind of failure points to a short list of likely causes. With this table, "something's wrong" becomes "check these three things first":

Symptom	Likely cause
`None != [...]`	missing `return`, or a method that mutates and returns None (`sort`, `append`)
`IndexError`	off-by-one in a loop bound, `while i < len(x)` vs `<=`, look-ahead `x[i+1]` at the end
`KeyError`	wrong key (str vs int, tuple order), key not inserted, deleted too early
`UnboundLocalError`	assignment to a name that should be `self.x` or that is only set in one branch
`RecursionError`	property setter assigning to itself; missing base case
test hangs	`while` loop that doesn't advance its index
result off by exactly 1	`range` bounds, `<` vs `<=`, 0- vs 1-based numbering
result is a permutation of expected	missing sort or wrong sort key / tie-break
works on ASCII, fails on 'é'	bytes vs chars confusion (`len(s)` vs `len(s.encode())`)
test passes alone, fails in suite	shared state: class attribute, module-level global, mutable default

A few rows need explaining:

- **"result is a permutation of expected"** means you got the right items in the wrong order, like `[2, 1, 3]` instead of `[1, 2, 3]`. The logic is fine. The sorting or tie-breaking isn't.
- **"test passes alone, fails in suite"**: each test is supposed to start fresh (that's what `setUp` is for). If a test passes when run on its own but fails when run with the others, an earlier test left something behind, and that something must live *outside* the test's own objects: a class attribute, a module-level variable, or a mutable default argument (Day 4, section 2).
- **"test hangs"**: press Ctrl-C. The traceback shows the line the program was stuck on, usually inside a `while` loop. Then check that every path through that loop moves its index forward.

7 The toolbox: Python you'll meet in today's exercises

Today's exercises are about *finding* bugs, so you'll be reading code more than writing it. Here are the techniques that code uses, so that while debugging you can focus on what's wrong rather than on what a line means.

7.1 `if __name__ == "__main__":`

You'll see this at the bottom of ex01 (and of many Python files):

```
if __name__ == "__main__":
    print(moving_average([1, 2, 3, 4, 5], 2))
```

Python sets the variable `__name__` to `"__main__"` only in the file you **run directly** (`python ex01_pdb_drill.py`). When the same file is **imported**, for example by a test, `__name__` is the module's name instead, so the block is skipped. It's the standard place for "try it out" code that shouldn't run during the tests. It's also a handy place to put a breakpoint and step through a function by hand.

7.2 A sliding window sum

`moving_average` computes the average of every `k` consecutive numbers. Adding up `k` numbers again for every window would be wasteful. Instead, the code keeps a running `window` sum and, when the window moves one step right, **adds the number that enters and subtracts the one that leaves**:

```
nums = [1, 2, 3, 4, 5], k = 2
window = 1 + 2 = 3                          covers positions 0..1
i = 2: add nums[2], remove nums[0] -> 4     covers positions 1..2
i = 3: add nums[3], remove nums[1] -> 7     covers positions 2..3
```

When the new window ends at position `i`, the number that just left is at position `i - k`. Getting that index one off is exactly the bug you'll chase with pdb. (Drill 1 is entirely about this technique.)

7.3 Sorting lists of pairs

`sorted` on a list of lists or tuples compares them **element by element**: by the first element, and only when those are equal, by the second (Day 1, section 6):

```
>>> sorted([[5, 6], [1, 3], [1, 2]])
[[1, 2], [1, 3], [5, 6]]
```

So `sorted(intervals)` sorts intervals by their start, which is exactly what merging needs.

7.4 Changing the last item of the result: `result[-1][1]`

When merging intervals, `result` is a list of `[start, end]` lists. `result[-1]` is the last merged interval, and `result[-1][1]` is its end. Read the double index from left to right: first pick the interval, then pick its end. Because each interval is a list, you can change its end in place: `result[-1][1] = new_end`. The merge step then is: "if the next interval starts before (or exactly where) the last one ends, stretch the last one; otherwise start a new one". Stretch to `max(old_end, new_end)`, not simply `new_end`: an interval entirely inside the previous one, like `[2, 3]` after `[1, 10]`, must not shrink it. `result and ...` at the start of the condition guards against the empty list: `result[-1]` on an empty list would raise `IndexError`, but `and` stops at the first false value, so the index is never evaluated when `result` is empty.

7.5 Money as whole cents

Floats can't store most decimal fractions exactly (Day 1, section 2): `0.1 + 0.2` isn't exactly `0.3`. Code that handles money therefore stores **integer cents**: 150.25 is stored as `15025`. Text is converted to cents when it's read, and back to text when it's printed. **Parsing** `"150.5"`: split at the dot with `partition` (Day 2), giving `"150"` and `"5"`. The fraction must count as **50** cents, not 5, so pad it on the right to two digits with `ljust(2, "0")` (Day 2) before converting. Then `cents = int(whole) * 100 + int(frac)`. A leading `-` is removed first and applied to the whole result at the end, `-cents if negative else cents`. Otherwise `"-0.05"` would come out as +5 cents, because the whole part `-0` is 0. **Formatting** `-310`: split off the sign, then `divmod(abs(cents), 100)` gives `(3, 10)` (Day 1, section 8). Use the absolute value, because `divmod` on a negative number rounds towards minus infinity and gives surprising results: `divmod(-310, 100)` is `(-4, 90)`. Finally, `f"{sign}{whole}.{frac:02d}"` writes the cents with **two** digits (`:02d` means "integer, at least 2 digits, padded with zeros", Day 2 section 4), so 5 cents prints as `0.05` and not `0.5`.

7.6 Parsing a line: comments, fields, and case

```
line = raw.split("#", 1)[0]     # everything before the first '#'
parts = line.split()            # fields separated by any whitespace
```

`raw.split("#", 1)` cuts at the first `#` into at most two pieces, and `[0]` takes the first piece. It works when there's no `#` too, because then the list has a single piece, the whole line. If `parts` is empty, the line was blank or only a comment. `action in ("BUY", "SELL")` checks one value against several allowed ones. The spec says actions are case-insensitive, so the code compares `parts[1].upper()`: whatever the user typed, `"buy"` or `"Buy"`, it becomes `"BUY"`.

7.7 A small list as a record

The portfolio stores each position as a two-item list `[qty, total_cost]`, so that it can be updated in place:

```
pos = self.positions.setdefault(symbol, [0, 0])   # Day 3, section 9
pos[0] += qty
pos[1] += cost
```

And it's read back by unpacking: `held, total_cost = self.positions[symbol]`. Unpacking works on any sequence of the right length, lists included, not just tuples.

7.8 Catching an error to add information, then raising again

The spec says every error message must start with the line number. The individual checks don't know the line number, but the loop does. So the loop catches the error, and raises a new one with the line number in front:

```
for n, raw in enumerate(lines, start=1):
    try:
        ...                                  # anything in here may raise ValueError
    except ValueError as e:
        raise ValueError(f"line {n}: {e}")
```

`except ValueError as e` catches it and names the exception object `e`. Putting `e` in an f-string gives its message, so `"not enough shares"` becomes `"line 4: not enough shares"`. `enumerate(..., start=1)` (Day 1) makes the line numbers start at 1, as humans count them.

7.9 Joining the report

A multi-line report is built as a list of lines and joined at the end with `"\n".join(lines)`, which puts a newline *between* lines and none after the last one (Day 2, section 5).

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 6` after every change; time `ex03` at 45 minutes.

File	Type	What
`exercises/ex01_pdb_drill.py`	**guided debug**	follow the walkthrough below using pdb
`exercises/ex02_intervals.py`	**debug**	interval utilities, 5 bugs
`exercises/ex03_ledger.py`	**debug, exam-style**	120-line portfolio ledger with a spec and 9 bugs. **Time it: 45 min.**

Background for ex02. An **interval** is a stretch of numbers from a start to an end, written `[start, end]`: for example, a meeting from hour 9 to hour 11 is `[9, 11]`. A typical task is **merging** overlapping intervals: if one meeting is `[1, 3]` and another `[2, 5]`, together they cover `[1, 5]`. The classic method is to sort the intervals by their start, then walk through them, and either extend the last merged interval (if the next one overlaps it) or start a new one. The bugs hide in the details the spec pins down. Do intervals that only *touch*, like `[1, 3]` and `[3, 5]`, count as overlapping? (In this exercise they do.) Is the input sorted first? Is the input list left unmodified? **Background for ex03.** A small investment **ledger**: it records buying and selling shares, and tracks your cash and how many shares of each company you hold. Its spec is at the top of the file. It's your first full exam rehearsal: apply the section 4 workflow step by step, and time yourself.

7.10 pdb walkthrough for ex01

The point of this walkthrough is to practise the *mechanics* of pdb on a small, contained bug, so you'll use it without hesitation in the exam. The first function computes a **moving average**: the average of every group of `k` neighbouring numbers. For `[1, 2, 3, 4, 5]` with `k = 2`, the groups are `[1, 2]`, `[2, 3]`, `[3, 4]`, `[4, 5]`, with averages 1.5, 2.5, 3.5, 4.5. Instead of re-adding each group from scratch, the function keeps a running total called `window`. At each step it adds the number that enters the group and subtracts the one that leaves. (This is the "sliding window" technique, covered in depth in drill 1.)

1. `python run.py 6 -k ex01` → `test_moving_average` fails with wrong numbers.
2. Open `exercises/ex01_pdb_drill.py`, add `breakpoint()` as the first line inside the `for` loop.
3. `cd day06_debugging_unittest/exercises && python -c "from ex01_pdb_drill import *; print(moving_average([1, 2, 3, 4, 5], 2))"`
4. At `(Pdb)`: `ll` (see the function), `p i, k, window`, `p nums[i], nums[i - k + 1]`. Ask: which element *leaves* the window when `nums[i]` enters? Compute it by hand for `i = 2, k = 2`.
5. `n` to execute the update line, `p window`. Compare with `sum(nums[i-k+1:i+1])` — type it at the prompt.
6. `q`, fix the line, remove `breakpoint()`, re-run the test.
7. Second bug, post-mortem style: run `python -m pdb -c continue ex01_pdb_drill.py` (the file has a `__main__` block that crashes). When it stops: `w`, `p line`, `p repr(line)`, `u` to see the caller. Fix it.

What each step teaches:

- **Step 3** calls the function directly with a tiny input, instead of going through the test, so you control exactly what goes in. (`python -c "..."` runs a one-line Python program given in quotes.)
- **Step 4** compares the program's idea of "the number that leaves the group" with yours.
- **Step 5** is the key move. You run one line, then compare the program's running total (`window`) with a total computed the slow, obviously-correct way (`sum(...)`), typed right at the pdb prompt. When they disagree, the line you just ran is the bug.
- **Step 7** practises post-mortem: the crash freezes the program, and `repr(line)` shows exactly what text the parser choked on, including any invisible characters.

```
python run.py 6
```

End-of-day checklist

Tick every box

- ☐ I can run a single test by its dotted name.
- ☐ I can set a conditional breakpoint and use `n`, `s`, `p`, `u`, `c`, `q`.
- ☐ I followed the 7-step workflow on ex03 and noted how long each bug took.

Exam lens

- Run all tests, triage by helper, and fix the lowest-level bug first.
- Re-run the full suite after every fix; a score drop is information, not failure.
- Time-box: 10–15 minutes without progress, write it down and move on.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.